

Programmazione 1

Corso c

04/10/2023

Operatori Relazionali e Booleani

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica

Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Operatori relazionali e valori booleani
- Operatori logici

Outline

- Operatori relazionali e valori booleani
- Operatori logici

Sintassi del C: operatori relazionali

- +

- -

- *

- /

- %

- ++

- --

- >

- <

- >=

- <=

- ==

- !=

- &&

- ||

- !

Cosa fanno gli operatori relazionali?

- Controllano che sia valida una **relazione** tra due espressioni:

```
int main(void) {  
    printf("%d\n", 3 > 4);  
    printf("%d\n", 3 >= 4);  
    printf("%d\n", 3 == 4);  
    printf("%d\n", 3 != 4);  
    printf("%d\n", 3 <= 4);  
    printf("%d\n", 3 < 4);  
}
```

Operatori relazionali

- In generale, un operatore relazionale verifica che esista una relazione tra due espressioni restituendo un intero:

Exp_1 **RELAZIONE** Exp_2

- Se **la relazione sussiste**, l'operatore restituisce **1**, altrimenti se la relazione **non sussiste**, restituisce **0**
- In C, per definizione, l'intero **0** rappresenta **il falso**, mentre **qualsiasi altro numero** intero rappresenta **il vero**

Cosa fanno gli operatori relazionali?

- Controllano che sia valida una **relazione** tra due espressioni:

```
int main(void) {  
    printf("%d\n", (3*3) > 4);  
    printf("%d\n", 19 >= (2+3+4+5));  
    printf("%d\n", 0 == 0);  
    printf("%d\n", 0 != 1);  
    printf("%d\n", 4 <= pow(4, 2));  
    printf("%d\n", sqrt(2) < 2);  
}
```

Cosa fanno gli operatori relazionali?

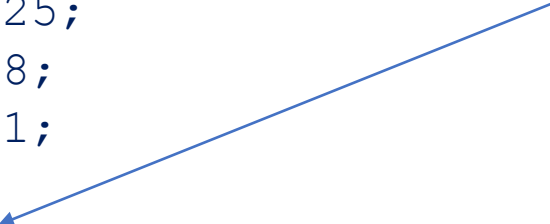
- Controllano che sia valida una **relazione** tra due espressioni:

```
int main(void) {  
    //un intero rappresenta un valore di verità  
    //-> variabile booleana  
    int valoreRelazione = 3 < 4;  
  
    printf("%d\n", valoreRelazione);  
}
```


Operatori relazionali

```
int main(void){  
    int votoScritto = 25;  
    int sogliaSuff = 18;  
    int sogliaLode = 31;  
  
    int scrittoSuperato = votoScritto > sogliaSuff;  
  
    printf("Lo studente supera lo scritto ? %d\n", scrittoSuperato);  
    printf("Lo studente prende la lode ? %d\n", votoScritto >= sogliaLode);  
}
```

scrittoSuperato è una variabile booleana



```
$ ./boolean  
Lo studente supera lo scritto ? 1  
Lo studente prende la lode ? 0
```

Il tipo bool

- Le ultime versioni dello standard prevedono un tipo specifico **bool**, usato per rappresentare un valore di verità. È equivalente a un **int** ristretto a soli 2 valori (n.b. in generale useremo **int**)

```
#include <stdbool.h>
int main(void) {
    //un bool rappresnta un valore di verità
    bool valoreRelazione = 3 < 4;

    printf("%d\n", valoreRelazione);
}
```

Il tipo bool

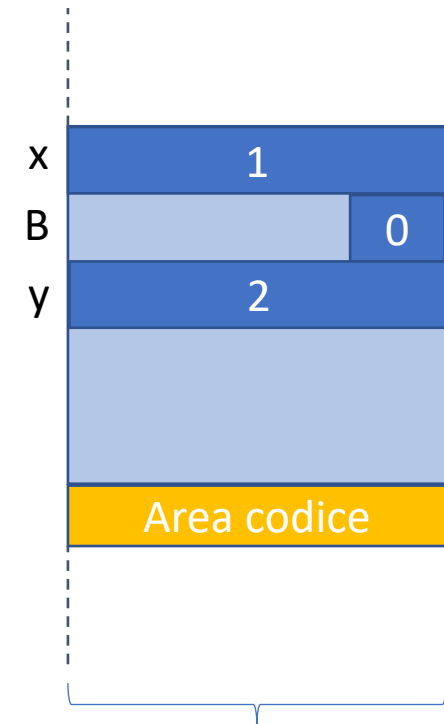
- L'header `stdbool.h` offre alcune utili funzionalità
 - Definisce il tipo non nativo **bool**
 - Definisce le costanti *true* e *false* utilizzabili al posto di 0 ed 1
- Richiede

```
#include <stdbool.h>
```

Il tipo bool

```
#include <stdio.h>
#include <stdbool.h>
```

```
int main() {
    int x = 1;
    bool B = false;
    int y = 2;
}
```



Simulatore Stack: <https://pythontutor.com/render.html#mode=display>

Outline

- Operatori relazionali e valori booleani
- Operatori logici

Valori booleani e operatori logici

- È necessario poter codificare **condizioni complesse fra diversi valori booleani**, come ad esempio:
 - entrambe sono vere
 - almeno una o entrambe sono vere
 - una sola è vera
- **Operatori logici** (o booleani)

Sintassi del C: operatori booleani

- +
- -
- *
- /
- %

- ++
- --

- >
- <
- >=
- <=
- ==
- !=

- &&
- ||
- !

Operatori booleani: && (AND)

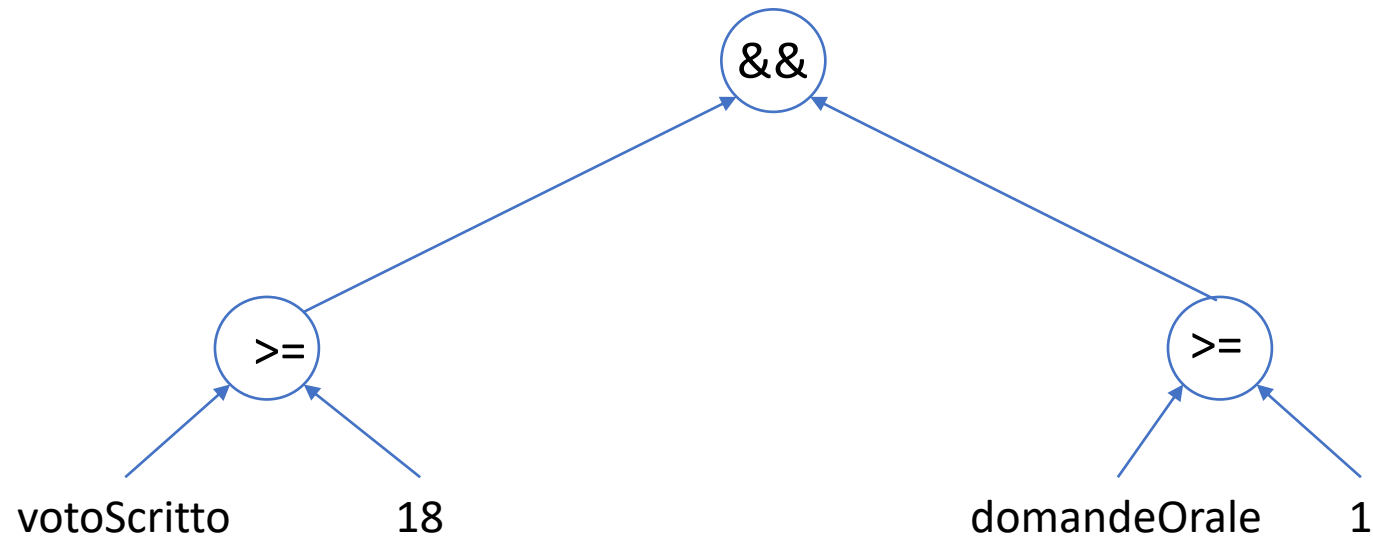
- *Lo studente supera l'esame se il voto dello scritto è almeno 18 e se risponde ad almeno 1 domande su 3 all'orale obbligatorio*
- E': votoScritto >= 18
- E'': domandeOrale >= 1
- E = E' AND E''
- Che si traduce nell'espressione con assegnamento

```
int superato = votoScritto >= 18 && domandeOrale >= 1
```

cioè superato vale 1 se E' vale 1 ed anche E'' vale 1

Operatori booleani: && (AND)

```
int superato = votoScritto > 18 && domandeOrale >= 1
```



Operatori booleani: && (AND)

- La condizione logica AND è codificata dall'operatore &&

E'	E''	E = E' && E''
0	0	0
0	1	0
1	0	0
1	1	1



Operatori booleani: || (OR)

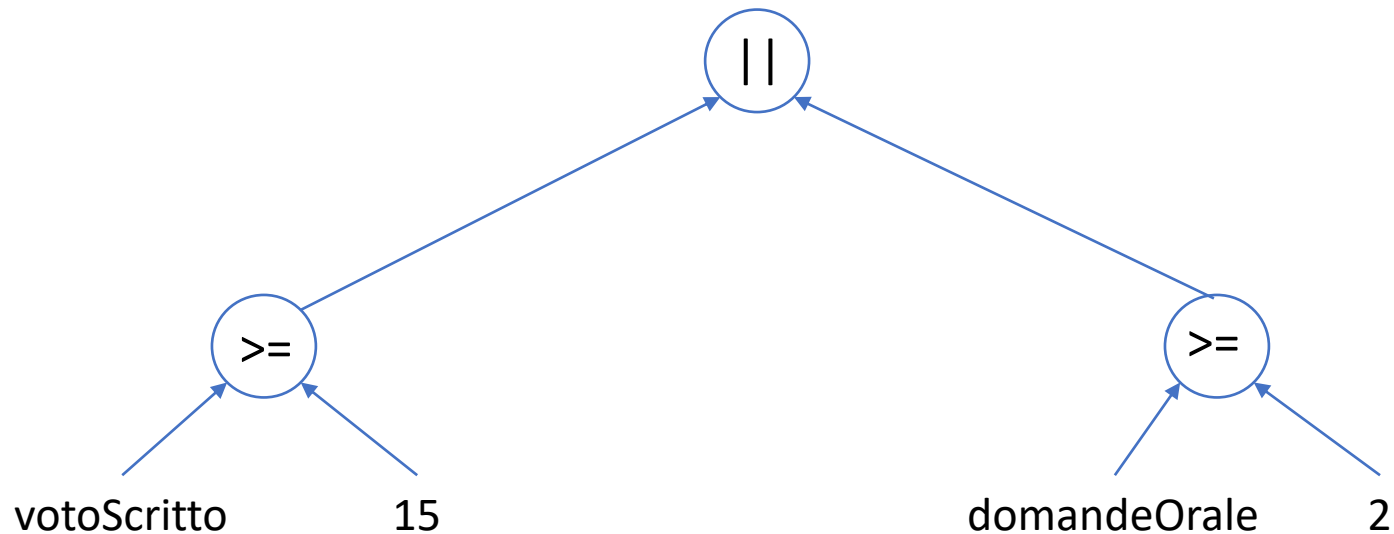
- *Lo studente supera l'esame se il voto dello scritto è almeno 15 oppure se risponde ad almeno 2 domande su 3 all'orale obbligatorio*
- E': votoScritto >= 15
- E'': domandeOrale >= 2
- E = E' OR E''
- Che si traduce nell'espressione con assegnamento

```
int superato = votoScritto >= 15 || domandeOrale >= 2
```

cioè superato vale 1 se E' vale 1 oppure E'' vale 1, o entrambe

Operatori booleani: || (OR)

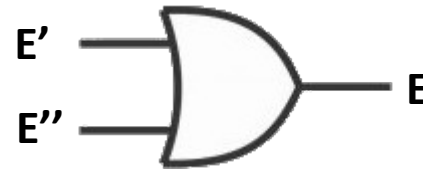
```
int superato = votoScritto > 15 || domandeOrale >= 2
```



Operatori booleani: || (OR)

- La condizione logica OR è codificata dall'operatore ||

E'	E''	E = E' E''
0	0	0
0	1	1
1	0	1
1	1	1



&& e ||

```
int main(void) {  
    int x = 5;  
    int y = 0;  
    printf("x>0&&y!=0:%d\n", x>0&&y!=0);  
    printf("x>0||y!=0:%d\n", x>0||y!=0);  
}
```

Operatori booleani: ! (NOT)

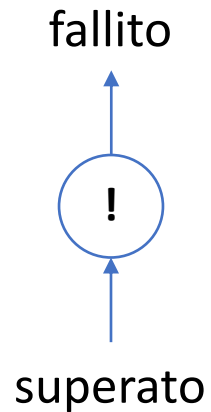
- *Lo studente fallisce se non supera l'esame*
- E': superato
- $E = \text{NOT } E'$
- Che si traduce nell'espressione con assegnamento

```
int fallito =! superato
```

cioè fallito vale 1 se E' vale 0

Operatori booleani: ! (NOT)

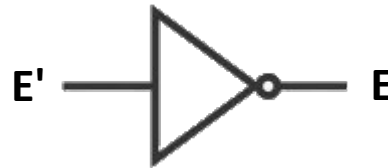
```
int fallito =! superato
```



Operatori booleani: ! (NOT)

- La condizione logica NOT è codificata dall'operatore !

E	E'
0	1
1	0



&& , || , !

```
int main(void) {  
    int x = 5;  
    int y = 0;  
    printf("x>0&&y!=0:%d\n", x>0&&! (y!=0) );  
    printf("x>0||y!=0:%d\n", x>0||! (y!=0) );  
}
```

Precedenza degli operatori

- Parentesi ()
- Negazione !
- Aritmetici * / %
- Aritmetici + -
- Relazionali < <= > >=
- Confronto == != prop. commutativa, associativa
- Logici && || prop. commutativa , associativa
- Assegnazione =

Espressioni booleane complesse

```
int main() {  
    int C, P, S;  
    printf("Chiave inserita (0/1):");  
    scanf("%d", &C);  
    printf("Pulsante premuto (0/1):");  
    scanf("%d", &P);  
    printf("Segnale sicurezza (0/1):");  
    scanf("%d", &S);  
    printf ("La macchina si avvia: %d\n", C && P && !S);  
}
```

```
$ ./macchina  
Chiave inserita (0/1):1  
Pulsante premuto (0/1):1  
Segnale sicurezza (0/1):0  
La macchina si avvia: 1
```

Variabili booleane

- Per ora ci siamo limitati a stampare il valore delle variabili booleane (che assumono solo valori 0 o 1)
- Vale sia per le variabili di input che per il valore di complesse espressioni relazionali-logiche
- Prossimamente vedremo come sfruttare tali variabili per scrivere programmi non strettamente sequenziali